

A Portable GPU Implementation of Boltzmann Particle Hydrodynamics with Massively

Akira Hayakawa

July 18, 2026

Abstract

Particle methods provide complementary routes to computational fluid dynamics. Smoothed Particle Hydrodynamics (SPH) discretizes continuum equations through kernel-weighted interactions among neighboring material particles, whereas Boltzmann Particle Hydrodynamics (BPH) represents a velocity distribution with Monte Carlo particles and alternates free flight with stochastic relaxation in Cartesian cells. BPH is attractive for high-Mach-number flow and vacuum, but its sampling fluctuation demands large particle populations and therefore high memory throughput. This paper presents `bph-gpu`, a portable GPU implementation in Rust built on `massively` v0.80 and the CubeCL kernel language. The central implementation problem is that relaxation is local to cells whose particle populations are irregular and change after every free-flight step, whereas the updated state must remain aligned and be written per particle. We resolve this mismatch by composing sort-by-key, reduce-by-key, scatter, gather, masked transform, and stream compaction. These operations turn cell-local relaxation into regular particle and cell arrays without a serial loop over cells or contended atomic accumulation. We derive the relaxation update, identify the ordering and data-layout invariants required by the GPU realization, and give a primitive-level algorithm detailed enough to reimplement it. Operator checks and a Sod shock tube calculation serve as a compact implementation demonstration. The focus is the GPU mapping rather than a new physical validation of BPH or a benchmark claim about a particular device.

Keywords: Boltzmann Particle Hydrodynamics, Smoothed Particle Hydrodynamics, GPU, Rust, parallel primitives, Monte Carlo particle method

1 Introduction

Particle methods have a long history in computational fluid dynamics. SPH was introduced independently by Lucy and by Gingold and Monaghan for astrophysical gas dynamics [1, 2]. It represents a continuum with moving material particles and approximates fields and their gradients by compact-support kernel sums over neighboring particles. The resulting Lagrangian, mesh-free description is particularly useful for large deformation, moving interfaces, and free surfaces [3].

A complementary line of development starts from kinetic theory. Direct Simulation Monte Carlo represents the molecular velocity distribution with simulator particles and separates deterministic motion from stochastic collisions [4]. Isaka, Matsuda, and Boffin adapted this idea to continuum gas flow as Molecular Hydrodynamics (MH), including internal degrees of freedom, shocks, vacuum, and high-Mach-number states [5]. Murata et al. subsequently analyzed the stability and time-step behavior of the continuum particle scheme [6]. Later work published in the proceedings of the Japan Society of Fluid Mechanics used the BPH designation and investigated variable particle number and weight [7]. BPH, as considered in this paper, follows the same stream-relax structure but replaces explicit molecular collisions by a cell-local relaxation modeled after the BGK approximation to the Boltzmann equation [8].

SPH and BPH are therefore related as Lagrangian particle approaches but are not the same discretization. SPH particles act as moving quadrature points for the continuum equations and

communicate through smoothing kernels. BPH particles sample a phase-space distribution; fixed cells localize relaxation, and density, bulk velocity, and pressure are recovered as population moments. Free flight allows a BPH particle to traverse multiple cells in one update. This supports stable calculations beyond the usual explicit-grid CFL limit, although a large time step still degrades accuracy through splitting error and numerical viscosity [5, 6].

The particle representation is attractive for extreme states because density is obtained from a nonnegative particle count and thermal energy from nonnegative kinetic and internal energy. Its principal cost is Monte Carlo noise. For a cell containing N_g particles, ordinary sampling error scales as $O(N_g^{-1/2})$; reducing it by a factor of ten requires approximately one hundred times as many samples. Memory capacity and throughput therefore become central concerns.

GPU research on related particle methods establishes a useful computational pattern. GPU-accelerated SPH uses spatial cells, particle reordering, neighbor processing, and on-device integration to handle millions of particles [9]. All-device DSMC implementations likewise parallelize particle motion, cell indexing, collisions, and sampling [10]. BPH has the same broad opportunity for data parallelism, but its cell relaxation requires segmented aggregation and redistribution rather than SPH kernel-force evaluation or pairwise DSMC collision selection.

This work develops `bph-gpu` in Rust on top of `massively` v0.80 and CubeCL [11, 12]. Its subject is the algorithmic mapping required to execute BPH on a GPU. The paper makes three contributions. First, it states the mathematical update that must be preserved by a parallel implementation, including equal-mass and full-relaxation assumptions. Second, it identifies the GPU-specific obstacles: time-dependent cell membership, highly uneven cell populations, transitions between particle and cell data, empty cells, and the need to reorder every particle field consistently. Third, it shows how the public parallel primitives of `massively` express each obstacle as a portable composition over device-resident arrays. A shock tube is included only to demonstrate that the composed implementation executes the intended BPH cycle; it is not presented as a comprehensive validation campaign.

2 Theoretical Background

2.1 The BGK equation and operator splitting

Let $\mathbf{x}$ be position, $\mathbf{c}$ molecular velocity, $\mathbf{F}$ external acceleration, n number density, and f a normalized velocity distribution. The BGK equation is

$$\frac{\partial(nf)}{\partial t} + \mathbf{c} \cdot \nabla_{\mathbf{x}}(nf) + \mathbf{F} \cdot \nabla_{\mathbf{c}}(nf) = n \frac{f^{\text{eq}} - f}{\tau}, \quad (1)$$

where τ is the relaxation time [8]. The equilibrium surrogate f^{eq} used by BPH need not be Maxwellian. It is sufficient for it to be isotropic in velocity space and to preserve the mass, momentum, and total energy moments,

$$\int n f Q(\mathbf{c}) d\mathbf{c} = \int n f^{\text{eq}} Q(\mathbf{c}) d\mathbf{c}, \quad Q \in \{m, m\mathbf{c}, \tfrac{1}{2}m|\mathbf{c}|^2 + e\}. \quad (2)$$

The implementation studied here uses a spherical-shell distribution for the angular part of this equilibrium surrogate.

Equation (1) is split into transport and collision. Conceptually, one time step is

$$\text{streaming: } (\mathbf{x}, \mathbf{c}) \longrightarrow (\mathbf{x}^*, \mathbf{c}^*), \quad \text{relaxation: } f^* \longrightarrow f^{n+1}. \quad (3)$$

The numerical method repeats the order shown in Fig. 1: compute cell indices, reorder particles, relax, stream, and apply boundary conditions. This ordering differs only in time labeling from a move-then-relax description, because positions produced by one iteration are consumed by relaxation in the next. With no external force, the first-order streaming update is

$$\mathbf{x}^{n+1} = \mathbf{x}^n + \mathbf{c}^{n+1} \Delta t. \quad (4)$$

The numerical demonstrations in Section 4 use no external force.

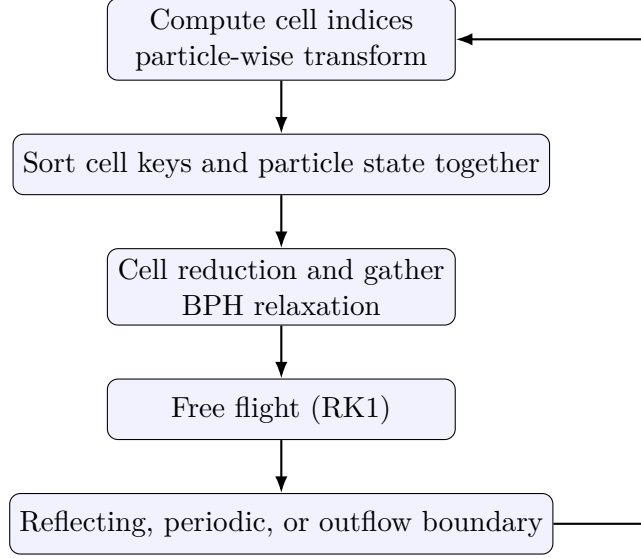


Figure 1: One BPH time step. In the GPU implementation, particle state remains resident on the device throughout the loop.

2.2 Particle state, heat-capacity ratio, and preconditions

Particle i has position $\mathbf{x}_i$, velocity $\mathbf{c}_i = (u_i, v_i, w_i)$, mass m_i , internal energy e_i , and cell index $g(i)$. In addition to three translational degrees of freedom, a particle has $s \geq 0$ internal degrees of freedom. The corresponding ratio of specific heats is

$$\gamma = 1 + \frac{2}{3+s} = \frac{s+5}{s+3}. \quad (5)$$

Thus $s = 0$ gives $\gamma = 5/3$, while $s = 2$ gives $\gamma = 7/5$.

The formulation considered here assumes that particles participating in a relaxation have equal masses and the same s . The arithmetic mean in Eq. (6) would have to be replaced by a mass-weighted mean for variable particle masses. The conservation derivation and numerical demonstrations below therefore assume $m_i = m$.

2.3 Relaxation within one cell

Let I_g be the set of particles in cell g and $N_g = |I_g|$. A cell with $N_g < 2$ cannot model a collision and is left unchanged. For $N_g \geq 2$, define the mean and thermal velocities as

$$\mathbf{U}_g = \frac{1}{N_g} \sum_{i \in I_g} \mathbf{c}_i, \quad \mathbf{q}_i = \mathbf{c}_i - \mathbf{U}_g. \quad (6)$$

The energy to be redistributed in the center-of-mass frame is

$$E_g = \sum_{i \in I_g} \left(\frac{1}{2} m |\mathbf{q}_i|^2 + e_i \right). \quad (7)$$

Equipartition assigns the post-relaxation translational kinetic and internal energies in the ratio $3 : s$,

$$K_g^* = \frac{3}{3+s} E_g, \quad E_{\text{int},g}^* = \frac{s}{3+s} E_g. \quad (8)$$

For each particle, two independent uniform variates $r_{1i}, r_{2i} \in [0, 1)$ produce

$$\mu_i = 1 - 2r_{1i}, \quad \theta_i = 2\pi r_{2i}, \quad (9)$$

$$\mathbf{d}_i = \left(\sqrt{1 - \mu_i^2} \sin \theta_i, \sqrt{1 - \mu_i^2} \cos \theta_i, \mu_i \right). \quad (10)$$

Because $|\mathbf{d}_i| = 1$, these vectors are uniformly distributed over the unit sphere. A finite sample does not have exactly zero vector sum, so it is balanced within each cell:

$$\hat{\mathbf{d}}_i = \mathbf{d}_i - \frac{1}{N_g} \sum_{j \in I_g} \mathbf{d}_j, \quad K_g^{\text{raw}} = \sum_{i \in I_g} \frac{1}{2} m |\hat{\mathbf{d}}_i|^2. \quad (11)$$

The velocity scale is

$$a_g = \sqrt{\frac{K_g^*}{K_g^{\text{raw}}}}, \quad (12)$$

and the relaxed state becomes

$$\mathbf{c}'_i = \mathbf{U}_g + a_g \hat{\mathbf{d}}_i, \quad e'_i = \frac{E_{\text{int},g}^*}{N_g}. \quad (13)$$

The rescaling in Eq. (12) requires $K_g^{\text{raw}} > 0$. With continuously sampled independent directions, the degenerate event has probability zero for $N_g \geq 2$, but a finite random generator can still represent it. The implementation guards the division and sets the scale to zero; a reimplementation that requires conservation in this exceptional case should resample the directions or supply a deterministic zero-mean shell configuration.

2.4 Conservation and numerical properties

Equation (11) gives $\sum_i \hat{\mathbf{d}}_i = \mathbf{0}$, and hence

$$\sum_{i \in I_g} m \mathbf{c}'_i = m N_g \mathbf{U}_g = \sum_{i \in I_g} m \mathbf{c}_i. \quad (14)$$

Cell momentum is therefore conserved. The cross term also vanishes, giving

$$\sum_{i \in I_g} \left(\frac{1}{2} m |\mathbf{c}'_i|^2 + e'_i \right) = \frac{1}{2} m N_g |\mathbf{U}_g|^2 + K_g^* + E_{\text{int},g}^* \quad (15)$$

$$= \frac{1}{2} m N_g |\mathbf{U}_g|^2 + E_g. \quad (16)$$

Thus, for $K_g^{\text{raw}} > 0$ and exact arithmetic, total energy is conserved as well. Mass conservation is immediate for cell relaxation because it neither creates nor removes particles. An outflow boundary may, of course, reduce the mass inside the computational domain.

A forward-Euler discretization of the BGK collision term permits a relaxation probability $P = \Delta t / \tau$. For viscous flow, relaxation can be sampled with this probability to represent a finite mean free path. Setting $P = 1$ instead produces the full-relaxation Euler model implemented by the current **bph-gpu** code; a finite- P Navier-Stokes viscosity model is not yet implemented.

The statement that BPH is not restricted by the CFL condition follows from particles being able to cross cell boundaries. It must not be interpreted as saying that every time step is accurate. Increasing Δt increases splitting error and numerical viscosity, and boundary processing must account consistently for every crossing made during a long flight.

3 GPU Mapping with Massively

3.1 Why cell relaxation is difficult on a GPU

Free flight is particle-local, but relaxation is neither a purely particle-local kernel nor a fixed-size cell stencil. Every particle must first discover its current cell. The number of particles in a cell may range from zero to a large fraction of the whole population, and the distribution changes at every step. Assigning one thread or one fixed workgroup to each cell therefore creates both load imbalance and an unknown local-storage requirement. Conversely, assigning one thread to each particle leaves the cell sums, means, and energy scale unavailable without a many-to-one aggregation.

The implementation must consequently alternate between two index spaces. The particle space has length N and is suitable for streaming, random sampling, and final state updates. The cell space has length K and contains populations, velocity sums, energies, and relaxation scales. A practical GPU algorithm must aggregate particle values into cells, represent empty cells, and redistribute cell values to particles while preserving the association among all structure-of-arrays columns. Table 1 summarizes how **massively** v0.80 supplies these transitions.

Table 1: BPH GPU challenges and their realization with **massively** v0.80.

BPH requirement	GPU difficulty	Massively composition
Form current cell groups	membership changes after streaming	<code>sort_by_key</code> with one shared permutation
Compute cell moments	uneven many-to-one aggregation	<code>reduce_by_key</code> on contiguous equal-key runs
Include empty cells	reductions emit occupied keys only	zero-filled cell array followed by <code>scatter</code>
Apply cell values to particles	results live in cell index space	key-indexed <code>gather</code> or an equivalent lazy indexed read
Update only collidable cells	cells below two particles must be unchanged	masked <code>transform</code> or <code>gather_where</code>
Remove outflow particles	the particle population is dynamic	<code>remove_where</code> with a boundary predicate

3.2 Execution model and data layout

bph-gpu implements BPH in Rust using the **massively** v0.80 parallel-algorithm library and the CubeCL kernel language [11, 12]. Particle data stay in device memory throughout the time-stepping loop. The runtime abstraction can lower the same algorithm to WGPU, CUDA, or HIP; the numerical results reported here use WGPU, while cross-backend equivalence and performance remain to be measured.

Particle state is stored in separate contiguous arrays for x, y, z, u, v, w, e , a structure-of-arrays (SoA) layout. Any operation that reorders particles computes one index permutation and applies it identically to every state column, including mass if it is not a global constant. Each physical column therefore remains contiguous for coalesced particle-wise access. The particle arrays and the cell-key array must always have the same length and particle ordering.

Cell arrays always have length K , including entries for empty cells. A reduction by key emits results only for nonempty cells, so the compact output is scattered by cell index into a dense, zero-initialized array. A gather then distributes each cell value back to its member particles. This reduction–scatter–gather cycle is the bridge between cell-local physical quantities and particle-level parallel work.

3.3 Cell keys and ordering invariants

Let N be the number of particles and let a Cartesian grid contain $K = n_x n_y n_z$ cells. For cell widths h_α and lower domain coordinates $x_{\alpha,\min}$, an interior particle receives

$$j_{\alpha i} = \left\lfloor \frac{x_{\alpha i} - x_{\alpha,\min}}{h_\alpha} \right\rfloor, \quad 0 \leq j_{\alpha i} < n_\alpha, \quad (17)$$

$$g_i = j_{xi} + n_x(j_{yi} + n_y j_{zi}), \quad 0 \leq g_i < K. \quad (18)$$

Periodic or reflecting coordinates must be returned to the domain, and outflow particles removed, before the next evaluation of Eq. (18). Keys from the previous free-flight step must not be reused.

The key–state pairs are sorted in ascending g_i , applying the resulting permutation to every particle field. Stability among equal keys is not a physical requirement; the required invariants are that equal keys form a contiguous run and that all fields retain their particle association. Spatial sorting is also a standard building block in GPU SPH and DSMC implementations [9, 10].

3.4 Massively primitive composition

The names in Table 1 correspond to the public algorithmic interfaces of `massively` v0.80: `sort_by_key`, `reduce_by_key`, `scatter`, `gather`, `lower_bound`, `upper_bound`, and masked transforms. The important point is their composition, not library-specific iterator or integer conversion types. The same decomposition can be reproduced in another GPU framework that provides operations with equivalent semantics.

One way to make the ordering explicit is to sort the pairs (g_i, i) by their first component. If p is the resulting permutation, then $g_{p_0} \leq g_{p_1} \leq \dots \leq g_{p_{N-1}}$, and every particle field is gathered in the order p . Sorting key–state records directly is equivalent, provided that all state fields receive exactly the same permutation. The ordering should be computed once per time step and shared by all fields.

The central reusable operation is a dense cell reduction. Given sorted keys $\mathbf{g}$ and particle values $\mathbf{a}$, reduction by key first returns unique occupied keys $\boldsymbol{\kappa}$ and compact sums $\mathbf{s}$. Scattering them into a zero-initialized cell array gives

$$(\boldsymbol{\kappa}, \mathbf{s}) = \text{reduce_by_key}(\mathbf{g}, \mathbf{a}), \quad \mathbf{A} = \mathbf{0}_K, \quad A_{\kappa_j} \leftarrow s_j. \quad (19)$$

This construction represents empty cells explicitly without a contended particle-to-cell atomic accumulation. The population of cell g is the difference between the upper and lower bounds of g in the sorted key array. Equivalently, it can be obtained by applying Eq. (19) to a particle array of ones.

For any dense cell field A_g , permutation by the particle keys constructs the particle field $a_i = A_{g_i}$. A GPU implementation may fuse this indexed read with its consuming kernel or materialize the gathered particle array; both have the same mathematical meaning. Vector quantities such as (u, v, w) may be processed component-wise or as tuples, provided that the SoA fields remain aligned. A mask for $N_g \geq 2$ selects particles eligible for relaxation, while particles in smaller cells retain their previous state.

3.5 Parallel relaxation

The following sequence is sufficient to reimplement one full-relaxation BPH collision step. Here `DENSEREDUCE` denotes Eq. (19), $\text{GATHER}(\mathbf{A}, \mathbf{g})_i = A_{g_i}$, and all particle maps are parallel.

1. Compute N_g and $\mathbf{S}_g = \text{DENSEREDUCE}(\mathbf{g}, \mathbf{c})_g$, then form $\mathbf{U}_g = \mathbf{S}_g / N_g$ for nonempty cells.
2. Gather $\mathbf{U}_{g_i}$, save it until the end of relaxation, and center each velocity: $\mathbf{q}_i = \mathbf{c}_i - \mathbf{U}_{g_i}$.
3. Compute $E_g = \text{DENSEREDUCE}(\mathbf{g}, \frac{1}{2}m|\mathbf{q}|^2 + e)_g$.
4. Generate two independent uniform particle arrays and transform them by Eq. (10) to obtain $\mathbf{d}_i$.

5. Compute $\bar{\mathbf{d}}_g = \text{DENSEREDUCE}(\mathbf{g}, \mathbf{d})_g / N_g$ and set $\hat{\mathbf{d}}_i = \mathbf{d}_i - \bar{\mathbf{d}}_{g_i}$. This finite-sample correction, rather than isotropy only in expectation, is what enforces discrete momentum conservation.
6. Compute $K_g^{\text{raw}} = \text{DENSEREDUCE}(\mathbf{g}, \frac{1}{2}m|\hat{\mathbf{d}}|^2)_g$, followed by K_g^* , $E_{\text{int},g}^*$, and a_g from Eqs. (8) and (12).
7. For cells with $N_g \geq 2$, gather these cell values and write $\mathbf{c}'_i = \mathbf{U}_{g_i} + a_{g_i}\hat{\mathbf{d}}_i$ and $e'_i = E_{\text{int},g_i}^* / N_{g_i}$. Leave particles in cells with $N_g < 2$ unchanged, and guard $K_g^{\text{raw}} = 0$ before the square root.

After relaxation, Eq. (4) advances positions and the selected boundary operator is applied. The next step then recomputes and sorts the cell keys. Independent random streams are required for the two angular variates; changing the streams between time steps avoids repeating the same angular sample. A deterministic seed schedule is useful for repeatable tests, although bitwise identity across backends is not assumed because random generators and floating-point reduction order may differ.

3.6 Memory traffic and performance considerations

Particle parallelism avoids the load imbalance of assigning one thread to a cell whose population may range from zero to many thousands. It does not make the algorithm free of global data movement. Each time step performs several full-particle passes, while sorting applies a permutation to every dynamic particle column. A comparison sort has $O(N \log N)$ work, and relaxation requires multiple reductions and particle-cell mappings. These stages are therefore the principal candidates for profiling, but no timing attribution is claimed without measurements of this implementation.

Fusing indexed reads with their consuming kernels can avoid some intermediate arrays, but several stages still require temporary N - or K -element buffers. Kernel fusion, persistent buffer reuse, a bounded-integer cell sort, and sort-free aggregation are thus the principal optimization candidates. The asymptotic additional storage is $O(N + K)$.

4 Implementation Demonstration

The implementation is exercised at two levels. Operator checks verify that segmented reductions produce the prescribed dense cell sums, including zeros for empty cells; one shared sort permutation keeps particle columns aligned; subtracting and restoring the gathered cell velocity recovers the original velocity; and the relaxation composition preserves the intended energy split for nondegenerate shell samples. These checks target the dataflow invariants on which the GPU mapping depends.

A Sod shock tube [13] provides a compact end-to-end demonstration. The unit interval is divided into $K = 2,000$ cells with $\Delta t = \Delta x = 5 \times 10^{-4}$. The initial left and right states have density ratio 8 : 1, pressure ratio 10 : 1, and zero bulk velocity. They are represented by 4,000 and 500 equal-mass particles per cell, respectively, for $N = 4,500,000$ particles. The calculation uses $s = 2$ ($\gamma = 7/5$), full relaxation, single-precision state, the WGPU backend, and advances to $t = 0.15$. Density is reported as the cell population divided by the initial left-state population of 4,000.

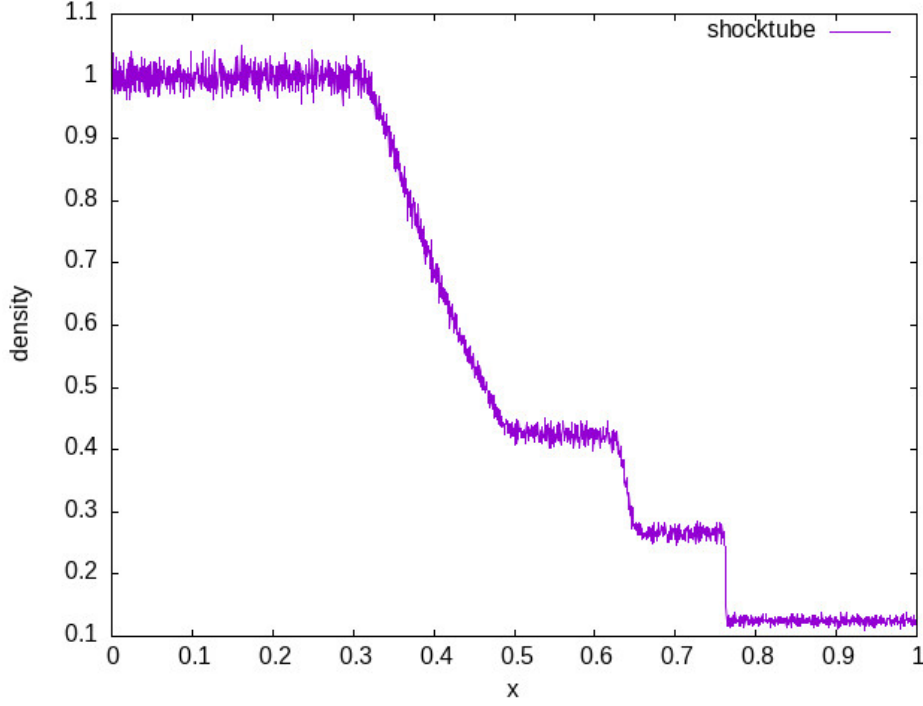


Figure 2: Normalized cell density for the Sod shock tube computed by **bph-gpu**. The rarefaction, contact discontinuity, and shock remain visible in the Monte Carlo particle density.

The purpose of Fig. 2 is limited: it confirms that cell-key generation, shared reordering, segmented relaxation, streaming, and boundary processing compose into a complete calculation at multi-million-particle scale. This technical paper does not use the plot to claim a convergence order, an error bound, or a speedup over another implementation. Likewise, backend-to-backend performance measurements are outside its present scope.

5 Conclusion

The main obstacle to a GPU implementation of BPH is the mismatch between particle-local execution and relaxation over irregular, time-dependent cell groups. **bph-gpu** resolves this mismatch with **massively** v0.80: sort-by-key forms contiguous cell segments while a shared permutation preserves SoA alignment; reduce-by-key and scatter construct dense cell moments; gather maps them back to particles; and masked transforms and compaction implement collision and boundary predicates. No cell-sized serial loop or particle-to-cell atomic accumulation is required.

The relaxation derivation identifies the quantities that this dataflow must preserve, while the primitive-level sequence specifies the algorithm independently of auxiliary library types. Operator checks cover its ordering and aggregation invariants, and a 4.5-million-particle Sod calculation demonstrates the complete GPU-resident cycle. Within the deliberately limited empirical scope of this technical paper, the result is a reimplementable parallel decomposition of BPH that can be lowered through CubeCL to multiple GPU runtime families.

References

- [1] L. B. Lucy, “A Numerical Approach to the Testing of the Fission Hypothesis,” *The Astrophysical Journal*, Vol. 82, pp. 1013–1024, 1977. <https://doi.org/10.1086/112164>

- [2] R. A. Gingold and J. J. Monaghan, “Smoothed Particle Hydrodynamics: Theory and Application to Non-Spherical Stars,” *Monthly Notices of the Royal Astronomical Society*, Vol. 181, No. 3, pp. 375–389, 1977. <https://doi.org/10.1093/mnras/181.3.375>
- [3] J. J. Monaghan, “Smoothed Particle Hydrodynamics,” *Annual Review of Astronomy and Astrophysics*, Vol. 30, pp. 543–574, 1992. <https://doi.org/10.1146/annurev.aa.30.090192.002551>
- [4] G. A. Bird, *Molecular Gas Dynamics and the Direct Simulation of Gas Flows*. Oxford University Press, 1994.
- [5] H. Isaka, T. Matsuda, and H. M. J. Boffin, “Application of Molecular Hydrodynamics to Astrophysical Flows: Bondi–Hoyle–Lyttleton Accretion Flow,” *Progress of Theoretical Physics*, Vol. 116, No. 6, pp. 1069–1096, 2006. <https://doi.org/10.1143/PTP.116.1069>
- [6] H. Murata, T. Matsuda, H. Isaka, Y. Ohsugi, and H. M. J. Boffin, “Application of Molecular Hydrodynamics to Astrophysical Flows. II: Unconditional Stability,” *Progress of Theoretical Physics*, Vol. 120, No. 2, pp. 347–369, 2008. <https://doi.org/10.1143/PTP.120.347>
- [7] H. Isaka, T. Matsuda, and Y. Ohsugi, “Improved Boltzmann Particle Method: Variable Number of Particles,” *Proceedings of the Japan Society of Fluid Mechanics Annual Meeting 2010*, p. 204, 2010 (in Japanese). <https://cir.nii.ac.jp/crid/2051996266952625024>
- [8] P. L. Bhatnagar, E. P. Gross, and M. Krook, “A Model for Collision Processes in Gases. I. Small Amplitude Processes in Charged and Neutral One-Component Systems,” *Physical Review*, Vol. 94, pp. 511–525, 1954. <https://doi.org/10.1103/PhysRev.94.511>
- [9] A. J. C. Crespo, J. M. Dominguez, A. Barreiro, M. Gómez-Gesteira, and B. D. Rogers, “GPUs, a New Tool of Acceleration in CFD: Efficiency and Reliability on Smoothed Particle Hydrodynamics Methods,” *PLOS ONE*, Vol. 6, No. 6, e20685, 2011. <https://doi.org/10.1371/journal.pone.0020685>
- [10] C.-C. Su, M. R. Smith, F.-A. Kuo, J.-S. Wu, C.-W. Hsieh, and K.-C. Tseng, “Large-Scale Simulations on Multiple Graphics Processing Units (GPUs) for the Direct Simulation Monte Carlo Method,” *Journal of Computational Physics*, Vol. 231, No. 23, pp. 7932–7958, 2012. <https://doi.org/10.1016/j.jcp.2012.07.038>
- [11] A. Hayakawa, “massively: Multi-platform GPU parallel algorithms for Rust,” version 0.80, software library, 2026. <https://github.com/akiradeveloper/massively>
- [12] Tracel AI, “CubeCL: Multi-platform high-performance computing language extension for Rust,” source code. <https://github.com/tracel-ai/cubec1>
- [13] G. A. Sod, “A Survey of Several Finite Difference Methods for Systems of Nonlinear Hyperbolic Conservation Laws,” *Journal of Computational Physics*, Vol. 27, No. 1, pp. 1–31, 1978. [https://doi.org/10.1016/0021-9991\(78\)90023-2](https://doi.org/10.1016/0021-9991(78)90023-2)